

## 5.1 Security Attacks like CSRF, XSS, SQL

### • Why we need to secure web-applications :

- Security is required to protect applications and user data from unauthorized access and malicious attacks.
- It ensures that only authenticated users can access the system and only authorized users can perform specific actions.
- In simple words:
  - Imagine your application is like a **house**.
    - **Authentication** = Checking who is entering the house.
    - **Authorization** = Deciding which rooms they can enter.
    - **Security attacks** = Different ways burglars try to break into your house.
  - Spring Security helps protect your application from these attacks.

### • Different types of attacks ?

- SQL injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

### • SQL injection:

- It is an attack where a hacker inserts **malicious SQL code** into **user input** to manipulate your database.
- The attacker enters **SQL commands** as input instead of entering normal data.
- **Example:** Database query is something like this, it will return the user if username and password matches.

```
String query =  
"SELECT * FROM users WHERE username = '" + username +  
"' AND password = '" + password + "'";
```

- What if the user enters malicious SQL instead of normal input ?

| User input                                                | SQL query according to input                                                           | Output                                                            |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| username : "sibasundarj8"<br>password : "xyz@123"         | SELECT * FROM users WHERE username = 'sibasundarj8'<br>AND password = 'xyz@123'        | User details. ✓<br>bcz username and password matches              |
| username : "sibasundarj8"<br>password : "' OR '1'='1'-- " | SELECT * FROM users WHERE username = 'sibasundarj8'<br>AND password = ' ' OR '1' = '1' | User details. ✓<br>bcz password returns true 1 = 1 -> <b>true</b> |

- In vulnerable applications, an attacker may also read, modify, or delete data, and in some cases execute statements like '**DROP TABLE**' if multiple statements are permitted.

## 5.1 Security Attacks like CSRF, XSS, SQL

- **How Spring Prevents It:**

- Spring Data JPA and Prepared Statements automatically separate the SQL query from user input.
- User values are sent as parameters rather than being concatenated into the SQL string.

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <b>Java:</b>      | SELECT * FROM product_table WHERE title_x = :title               |
| <b>Hibernate:</b> | select p1.id,p1.title_x,from product_table p1 where p1.title_x=? |

- **Interview Answer:** SQL Injection is an attack where malicious SQL statements are injected into user input to manipulate the database. It is prevented by using parameterized queries, prepared statements, or ORM frameworks like Spring Data JPA.

- **Cross-Site Scripting (XSS) :**

- **Cross-Site Scripting (XSS)** occurs when an application includes untrusted user input in a webpage without proper escaping or sanitization, allowing attacker-controlled JavaScript to execute in another user's browser.
- In simple words, The attacker tricks **your website** into running **their JavaScript** in the **browser of users visiting your website**.
- **Example:** Suppose your website allows comments.

| User Comment                                | Output                                                                              |
|---------------------------------------------|-------------------------------------------------------------------------------------|
| Comment: "Nice project!"                    | Displays "Nice project!" just as normal string nothing fancy.                       |
| Comment: "<script>alert('Hacked')</script>" | Browser executes the script and every user visiting that page sees: "Hacked" pop-up |

- **Real attackers** can:
  - Steal sensitive data.
  - Perform actions on behalf of users.
  - Access tokens stored in JavaScript-accessible storage (such as localStorage).
  - Manipulate the page.
  - Steal cookies that are not protected with the **HttpOnly flag**.

- **Types of XSS :**

| Type          | Description                                                                   |
|---------------|-------------------------------------------------------------------------------|
| Stored XSS    | Malicious script is stored in the database and served to every visitor.       |
| Reflected XSS | Script comes from the request (URL, form) and is reflected immediately.       |
| DOM-based XSS | Vulnerability exists entirely in client-side JavaScript manipulating the DOM. |

- **How to Prevent XSS :**

- Escape HTML output
- Validate input (defense-in-depth)
- Use Content Security Policy (CSP)
- Avoid rendering raw HTML from users
- Spring Security provides security headers (such as Content Security Policy if configured), but **preventing XSS** primarily depends on proper output encoding, safe rendering of user input, and avoiding unsafe APIs.
- **Interview Answer:** Cross-Site Scripting (XSS) is an attack in which malicious JavaScript is injected into web pages and executed in another user's browser. It is prevented by escaping HTML, validating user input, using Content Security Policy, and avoiding rendering untrusted HTML.

## 5.1 Security Attacks like CSRF, XSS, SQL

### • Cross-Site Request Forgery (CSRF):

- **CSRF** tricks a **logged-in user** into performing an **action** they **never intended**. The attacker doesn't steal the password. Instead they misuse the user's already authenticated session.
- **Example:**
  - Suppose you're logged into your banking website.
  - Your browser already has: **Session Cookie**
  - Now you visit a malicious website which contains a hidden form that submits a request to your bank.

```
<!-- Hidden inside that page: -->
<form action="https://bank.com/transfer" method="POST">
  <input name="amount" value="100000">
  <input name="to" value="attacker">
</form>
<script>
  document.forms[0].submit();
</script>
```

- Your browser automatically sends **session cookie** because you're already logged into the bank.
  - The bank processes the transfer because it believes the request came from the authenticated user.
  - The attacker never stole your password or session, they just fooled **your browser** into sending a legitimate request.
- **How Spring Security Prevents It ?**
  - Spring Security requires a valid **CSRF token** for state-changing HTTP requests (POST, PUT, PATCH, DELETE).
- **Interview Answer:**
  - CSRF is an attack where a malicious website tricks an authenticated user's browser into sending unwanted requests to another website. Spring Security prevents this by generating and validating a unique CSRF token for every user session.

### Summary:

- SQL Injection, XSS, and CSRF are three common web security attacks.
- SQL Injection targets the database by injecting malicious SQL queries through user input and is prevented using parameterized queries or ORM frameworks.
- XSS targets user's browsers by injecting malicious JavaScript into web pages, which can steal cookies or manipulate the page; it is prevented through output escaping and input validation.
- CSRF targets authenticated users by tricking their browsers into sending unintended requests using existing session cookies. Spring Security provides built-in CSRF protection using CSRF tokens.
- Spring security also helps in mitigating XSS through security headers, while SQL Injection is primarily prevented through secure database access practices such as prepared statements and JPA.